

Neuronales Netz für den kalifornischen Hauspreis-Datensatz

Björn Becker^a^aJustus-Liebig-Universität Gießen, Matrikelnummer: ...

Gruppe 308 – KI I Projekt WS 2025/26

Zusammenfassung—Dieses Dokument fasst meinen Arbeitsfortschritt bei der Entwicklung Neuronaler Netze für die Vorhersage von Hauspreisen anhand des "California housing dataset" zusammen. Es werden vor allem die Hyperparameteroptimierung und Ensemble Erstellung behandelt. Das beste Modell erreicht einen R^2 -Wert von ... auf dem Testset und übertrifft damit die lineare Regression ($R^2 = 0,63$) deutlich.

1. FRAGESTELLUNG UND ZIEL

Ziel des Projekts war es, ein Neuronales Netz aufzubauen, welches die Hauspreise im "California housing dataset" vorhersagen kann. Neuronale Netze stellen eine zentrale Methode der modernen Künstlichen Intelligenz dar (Russell & Norvig, 2021). Das Ergebnis sollte mit einer linearen Regression verglichen werden, um den Mehrwert komplexerer Modelle zu bewerten. Aufgrund der zeitlichen Verfügbarkeit der Gruppenmitglieder hatte ich die Chance die Gruppenergebnisse zusammenzuführen und weiter zu verwenden. Daher lag der Fokus meiner Arbeit auf der Hyperparameteroptimierung durch die Random-Search-Methode und der Erstellung von Ensembles.

2. DARSTELLUNG DER ARBEITSPROZESSE

2.1. Vorbereitung

Vor unserem ersten Gruppentreffen habe ich mich mit der Aufgabe vertraut gemacht und bin die Vorlesungsfolien durchgegangen. Zusätzlich informierte ich mich über die Einrichtung einer gemeinsamen Programmierumgebung über GitLab. Allerdings kam im Verlauf des ersten Gruppentreffens auf, dass insgesamt in der Gruppe mehr Erfahrungen mit GitHub vorhanden waren, weshalb wir uns dafür entschieden. Florian übernahm die Einrichtung des Repositories und die Architektur der Ordnerstruktur (vgl. Logbuch von Florian). Zwischen dem ersten und 5. Gruppentreffen lag mehr als ein Monat, in dem ich lediglich an den Treffen teilnahm, Florian, Felix H. und Kolja bereits den wesentlichen Teil ihres Arbeitspensums erledigten. Nachdem ich mich einen Tag vor dem fünften Treffen mit dem Fortschritt der Gruppe und den im Repository zugänglichen Dateien, u.a. den Hilfsfunktionen für die Modellevaluation vertraut machte, machte ich mir anhand der Vorlesungsfolien Gedanken über mögliche Aufgaben, an denen ich arbeiten könnte. Außerdem habe ich via venv eine passende Python-Umgebung eingerichtet und die benötigten Pakete installiert, um die Jupyter-Notebooks der Gruppe ausführen zu können.

2.2. Erste Modelle

Nach dem Gruppentreffen, habe ich zuerst zwei eigene Neuronale Netze gebaut, um die Anwendung der Keras-Bibliothek zu erlernen und den Gebrauch einiger Hilfsfunktionen zu verstehen. Auf Grundlage der Gruppenergebnisse entschied ich mich bei allen weiteren Netzen für die Verwendung der Aktivierungsfunktion `relu` und des Optimierers `Adam` (vgl. Logbuch von Felix H.). Ich orientierte mich auch bei Hyperparameterwahl und Struktur des Netzes, d.h. kleiner werdenden Schichten, an den bereits erstellten Modellen der Gruppe. Zudem wurde aus den Gruppenergebnissen deutlich, dass nicht alle Features für die Vorhersage relevant sind, weshalb ich mich im Folgenden nur mit den Features `MMedInc`, `AveOccup`, `LLatitude` und `LLongitude` beschäftigte. Die Auswahl relevanter Features ist ein zentraler Bestandteil von Data-Mining-Prozessen (Runkler, 2012), was die Vorgehensweise bestätigt. Mit den Netzen erreichte ich R^2 -Werte

von 0.7188 und 0.7193, was den Vergleich mit der linearen Regression mit $R^2 = 0.7091$ (vgl. `01_Lasso_Ridge.ipynb`) bereits übertraf.

2.3. Random-Search

Nun beginnt der Hauptteil meiner Arbeit. Für alle Hyperparameter war von den Gruppenmitgliedern eine ungefähre optimale Größenordnung vorgegeben worden. Diese habe ich verwendet, um gute Modellkonfigurationen zu generieren. Problematisch erschien mir, dass bisher die Hyperparameter jeweils einzeln variiert wurden, was dazu führen kann, dass bei einer geordneten Suche nicht die optimale Kombination gefunden wird, da der Algorithmus eher Plateaus findet als optimale Kombinationen. Dies ist darauf zurückzuführen, dass Hyperparameter häufig Wechselwirkungen aufweisen und daher nicht unabhängig voneinander betrachtet werden sollten (Ertel, 2021). Daher entschied ich mich für die Random-Search-Methode, bei der die Hyperparameter zufällig aus den vorgegebenen Bereichen ausgewählt werden. Zu Beginn wurden diese gewählt, wie in Tabelle 1 dargestellt. Random Search gilt als effektive Methode zur Hyperparameteroptimierung, da sie im Vergleich zu systematischen Suchverfahren häufig schneller gute Kombinationen findet (vgl. Bergstra & Bengio, 2012).

Hyperparameter	Wertebereich
Anzahl Schichten	2, 3
Anzahl Neuronen pro Schicht	32, 64, 128
Lernrate	0.0001 bis 0.01
Dropout-Rate	0.1 bis 0.3
Batchgröße	16, 32, 64
Anzahl Epochen	50, 100
L2-Regularisierung	0.00001 bis 0.001
Aktivierungsfunktion	ReLU
Optimierer	Adam
EarlyStopping	Ja

Tabelle 1. Hyperparameter und Wertebereiche für die Random-Search

Damit die Rechenzeit im Rahmen blieb nutze ich Parallelisierung, womit mehrere Modelle gleichzeitig trainiert werden konnten. Aus mehreren hundert Modellen, wählte ich die besten 50 aus, um diese zu analysieren.

R^2 -Valuation	R^2 -Test	units	depth	learning_rate	l2_reg	dropout_rate	batch_size	epochs
0.757183	0.743349	128	2	0.000343	0.000052	0.156997	16	81
0.751369	0.736966	128	2	0.001588	0.000024	0.163947	16	66
0.750405	0.736433	128	2	0.000395	0.000017	0.256285	64	93
0.747135	0.733957	64	3	0.000941	0.000284	0.116963	16	77
0.746669	0.733977	128	3	0.001095	0.000016	0.195057	32	61

Tabelle 2. Ergebnisse der Random-Search: Hyperparameterkonfiguration einiger der besten Modelle.

Aus den Ergebnissen waren keine klaren Aussagen mit bloßem Auge möglich, weshalb ich mich dazu entschied einen Random-Forest-Classifer mit den Hyperparametern zu trainieren und damit die Hyperparameter-Importance (Abb. 1) bzw. den Einfluss der Hyperparameter auf die Modellleistung zu erzeugen. Außerdem ließ ich mir die im `sklearn`-Modul verfügbaren Partial-Dependence-Plots anzeigen, um die Auswirkungen der Hyperparameter auf die Modellleistung zu verstehen.

Mit den Partial-Dependence-Plots (Abb. 2) war es dann möglich, die Auswirkungen der Hyperparameter auf die Modellleistung zu verstehen. Durch ein Iteratives Vorgehen, bei welchem nach jeder Iteration, d.h. nach der Erstellung von einigen hundert Modellen, die Hyperparameter-Importance analysiert und die Wertebereiche der Hyperparameter angepasst wurden, konnte das Modell sukzessive

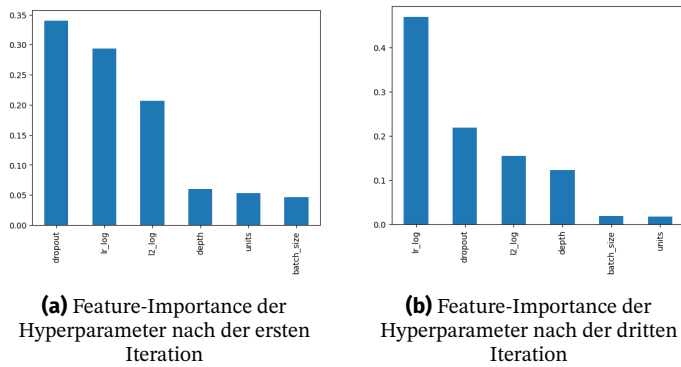
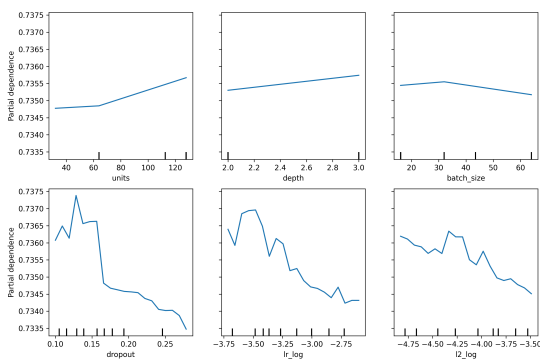
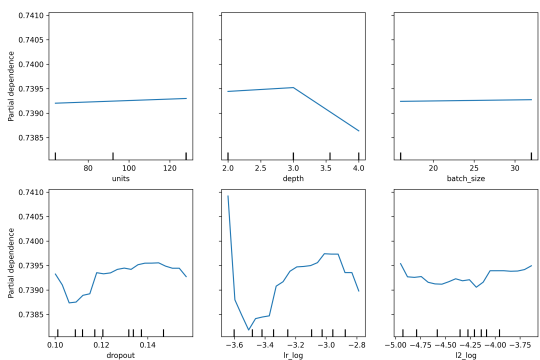


Abbildung 1. Vergleich der Hyperparameter-Importance nach der ersten und dritten Iteration der Random-Search.

verbessert werden. Allerdings zeigte sich nach der zweiten Iteration schon keine große Veränderung mehr, weshalb ich das Vorgehen nach der dritten Iteration beendete.



(a) Partial-Dependence-Plots nach der ersten Iteration



(b) Partial-Dependence-Plots nach der dritten Iteration

Abbildung 2. Partial-Dependence-Plots nach der ersten und dritten Random-Search-Iteration.

Währenddessen probierte ich auch größere Spannen bei den Hyperparametern aus, um zu sehen, ob es vielleicht doch noch bessere Modelle gibt. Zu nennen ist hier zum Beispiel die Tiefe des Netzes oder eine kleinere Lernrate. Ziel war es, etwas über den Tellerrand hinauszuschauen, um zu prüfen, ob es vielleicht doch noch bessere Modelle gibt. Es zeigte sich, dass die Tiefe des Netzes von 4 Layern gute Ergebnisse lieferte, weshalb ich diesen Wert in die Random-Search mit aufnahm. Vorerst nutze ich eine maximale Epochenzahl von 100. Da trotz Early-Stopping aber nach der zweiten Iteration die Netze noch nicht ausreichend trainiert worden waren, erhöhte ich die

maximale Epochenzahl. Schlussendlich passte ich die Hyperparameterbereiche folgendermaßen an (Tab. 3):

Hyperparameter	Wertebereich
Anzahl Schichten	2, 3, 4
Anzahl Neuronen pro Schicht	32, 64, 128
Lernrate	$10^{-3.3}$ bis 10^{-2}
Dropout-Rate	0.115 bis 0.16
Batchgröße	16, 32, 64
Anzahl Epochen	500
L2-Regularisierung	0.00001 bis 0.001
Aktivierungsfunktion	ReLU
Optimierer	Adam
EarlyStopping	Ja

Tabelle 3. Hyperparameter und Wertebereiche für die Random-Search

Das beste gefundene Netz erreichte einen R^2 -Wert von 0.752 auf dem Testset, was eine deutliche Verbesserung gegenüber der linearen Regression darstellt.

2.4. Ensemble

Nachdem ich 10 meiner besten Hyperparameterkonfigurationen aus der Random-Search-Phase ausgewählt hatte, entschied ich mich, ein Ensemble zu erstellen. Ensemble-Methoden werden eingesetzt, um durch die Kombination mehrerer Modelle die Generalisierungsfähigkeit zu verbessern (Kruse et al., 2013). Dieses Ensemble verbesserte den Score nicht, sondern erreichte lediglich einen R^2 -Wert von 0.7499, was trotzdem besser war als die allermeisten Einzelmodelle. Um eine Aussage über den gesamten California-Housing-Datensatz zu treffen, trainierte ich das Ensemble auf eben diesen und erreichte einen R^2 -Wert von 0.788, was eine deutliche Verbesserung gegenüber dem verkleinerten Datensatz darstellte.

2.5. Feature-Engineering

Aus dem deutlichen Sprung des R^2 durch die Hinzunahme der restlichen Features lag nahe, Feature-Engineering zu betreiben. Tatsächlich konnte ich durch die Erstellung neuer Features (siehe Tab. 4 u. Tab. ??) das Ensemble in zwei Versuchen nicht verbessern, sondern verschlechterte es sogar.

neues Feature	Definition
rooms per household	AveRooms/AveOccupancy
bedrooms per room	AveBedrms/AveRooms
population per household	Population/AveOccupancy
income per household	MedInc/AveOccupancy

R^2 -Wert 0.65

Tabelle 4. zusätzliche Features und deren Definitionen im ersten Versuch des Feature-Engineerings

neues Feature	Definition
income · rooms	MedInc × AveRooms
income · population	MedInc × Population
distance to center	$\sqrt{(\text{Latitude} - 36)^2 + (\text{Longitude} + 119)^2}$
squared Occupancy	AveOccup × AveOccup
logarithm of population	$\log(\text{Population})$

R^2 -Wert 0.68

Tabelle 5. zusätzliche Features und deren Definitionen im zweiten Versuch des Feature-Engineerings

2.6. Meine Erkenntnisse und Ausblick

Kern meiner Arbeit war die Hyperparameteroptimierung durch die Random-Search-Methode und Ensembles auf verschiedenen Datensätzen zu trainieren. In meiner Arbeit habe ich erkannt, dass es sehr schwierig ist, Neuronale Netze anhand von Hyperparameteroptimierung zu verbessern. Oft kommen Verbesserungen überraschend, wie

bei der Hinzunahme der als unwichtig deklarierten Features und manchmal haben Entscheidungen die logisch erscheinen, wie das daraus gefolgte Feature-Engineering, negative Auswirkungen auf die Güte des Netzes. Während meiner Arbeit habe ich eher gelernt, wie man sicher viele gute Netze generiert und weniger wie man ein sehr gutes erhält. Es konnte in meiner Arbeit gezeigt werden, dass Neuronale Netze vor allem mit Hyperparameteroptimierung durch Random-Search-Methoden die Zusammenhänge innerhalb des California-Housing-Datensatzes wesentlich besser erfassen können als lineare Regressionen. Weitere Verbesserungen könnten z.B. durch das Untersuchen der Architektur der Netze, mit Hilfe eines Verkleinerungsfaktors für die Schichten, erreicht werden.

2.7. Reflexion und Einordnung

Insgesamt konnte mein bestes Modell einen R^2 -Wert von 0.788 auf dem gesamten Datensatz erreichen, was für wirtschaftliche Anwendungen bereits als gutes Ergebnis interpretiert werden kann (Investopedia, o.J.), wobei diese Interpretation stark kontextabhängig ist. Online findet sich eine Ausarbeitung des Datensatzes, bei der ein Neuronales Netz mit einem R^2 -Wert von etwa 0.80 auf dem Testset erreicht wurde (medium, o.J.), was zeigt, dass das erreichte Ergebnis konkurrenzfähig ist, aber auch noch Raum für Verbesserungen bietet. Die Methodik der Hyperparameteroptimierung durch Random-Search-Methoden hat das Problem, dass es keine Garantie dafür gibt, dass die optimale Kombination gefunden wird. Es muss bei jedem Hyperparameter in einem kleinen Intervall gesucht werden, wobei nicht ausgeschlossen werden kann, dass die optimale Kombination außerhalb dieses Intervalls liegt. Die gebildeten Ensembles haben die Vorhersageleistung kaum verbessert. Als nächster Schritt sollte in das Ensemble andere Modelle neben Neuronalen Netzen aufgenommen werden, um die Diversität der Modelle zu erhöhen. Außerdem ist hat auch das Rauschen der Modelle einen Einfluss auf die Güte des Ensembles. Das Feature-Engineering hat sich als schwierig erwiesen und sich negativ auf die Güte des Netzes ausgewirkt. Möglicherweise könnten aber andere Features auch einen positiven Einfluss haben, weshalb es sich lohnen könnte, weitere Features zu erstellen und zu testen. Es sei noch erwähnt, dass der Datensplit in Trainings-, Validierungs- und Testset immer der gleiche war, um bessere Vergleichbarkeit zu gewähren. Allerdings sind die erzielten Ergebnisse nicht unbedingt auf alle anderen möglichen Splits übertragbar.

LITERATUR

- [1] Felix Hollfoth. *Logbuch – KI I Projekt, Gruppe 308*. Unveröffentlichtes Arbeitsdokument, 2026.
- [2] Florian Adamczyk. *Logbuch – KI I Projekt, Gruppe 308*. Unveröffentlichtes Arbeitsdokument, 2026.
- [3] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. 4th Edition, Pearson, 2021.
- [4] Wolfgang Ertel. *Introduction to Artificial Intelligence*. 3rd Edition, Springer, 2021.
- [5] Thomas Runkler. *Data Mining: Methoden und Algorithmen intelligenter Datenanalyse*. Vieweg+Teubner, 2012.
- [6] Rudolf Kruse, Christian Borgelt, Frank Klawonn, Christian Mowes, Matthias Steinbrecher. *Computational Intelligence: Eine methodische Einführung in Künstliche Neuronale Netze, Evolutionäre Algorithmen und Fuzzy-Systeme*. Springer, 2013.
- [7] James Bergstra and Yoshua Bengio. Random Search for Hyperparameter Optimization. *Journal of Machine Learning Research*, 13:281–305, 2012.
- [8] Investopedia. *R-Squared*. Verfügbar unter: <https://www.investopedia.com/terms/r/r-squared.asp> (Zugriff am 26.03.2026).

- [9] Advika California Housing Price Prediction: A Multi-Model Analysis Verfügbar unter: <https://medium.com/@saudhaminiupdated/california-housing-price-prediction-a-multi-model-analysis-e63bd9cbddb1> (Zugriff am 27.03.2026)